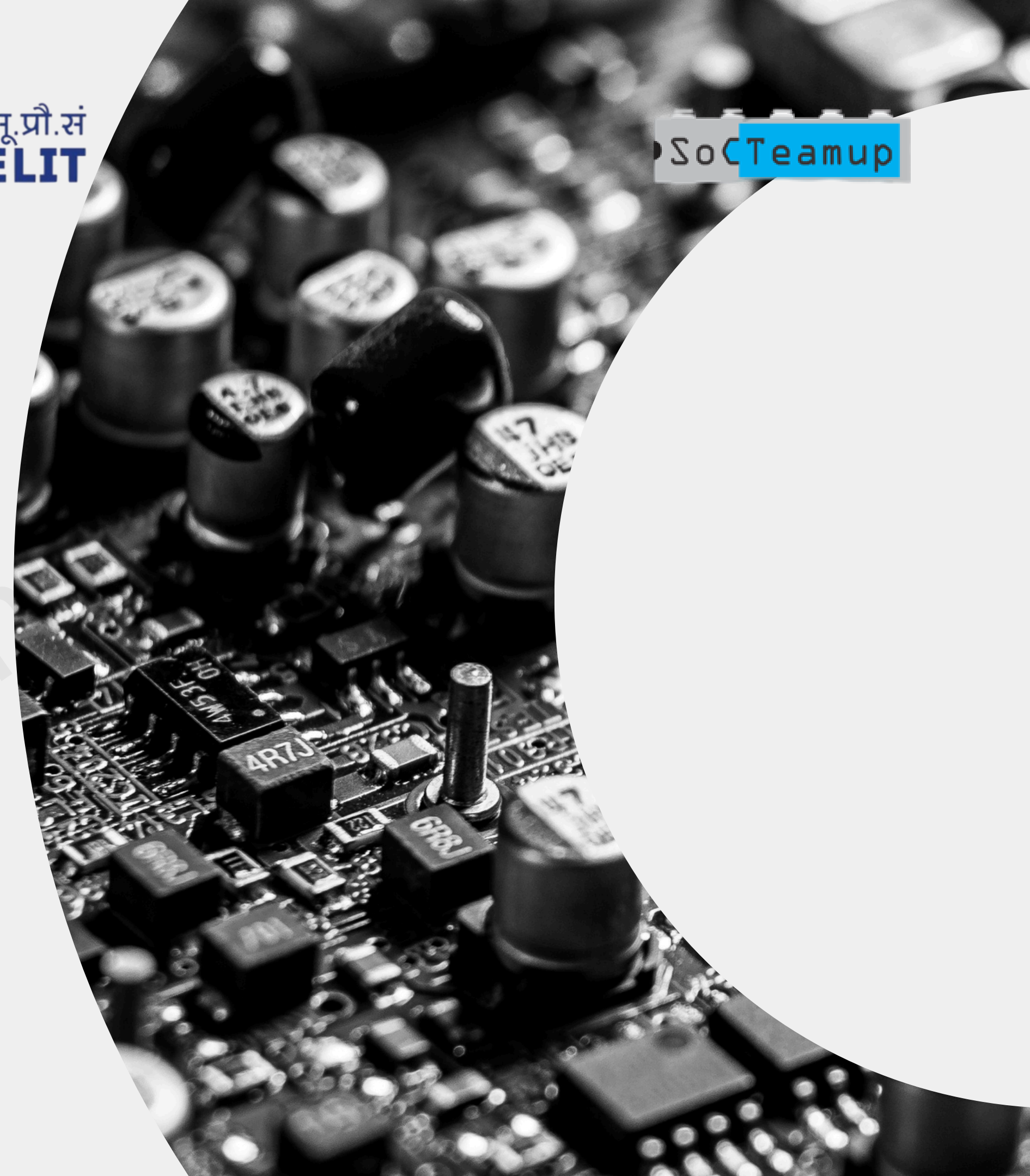


Synthesizable vs non-synthesizable constructs





Outline

What is Synthesizable Code?

**What is Non-Synthesizable
Code?**

**Common Synthesizable
Constructs**

**Common Non-Synthesizable
Constructs**

Real-World Examples

How to Check Synthesizability

Exercise: Fix the Design

Summary & Best Practices

➔ What is **Synthesizable Code**?

- Can be converted into gates by synthesis tools (**e.g., Yosys**)
- Maps to real hardware like flip-flops, muxes, logic gates

Example:

```
always @(posedge clk)  
q <= d; // Synthesizes to D Flip-Flop
```

What is **Non-Synthesizable** Code?

- Used for simulation or testbenches only
- Has no hardware equivalent

Example:

`$display()`, `#10 delay`, initial blocks
forever loops, `$stop`, `$finish`

Common Synthesizable Constructs

Construct	Purpose
always @(posedge clk)	Sequential logic(FFs)
assign	Combinational logic
if / case	Conditional logic(MUX)
wire / reg	Data connection/storage

➔ Common Non-Synthesizable Constructs

Construct	Use Case	Why Not Synthesizable
#10	Timing delays	No real equivalent in silicon
\$display	Console output	Debug only
Initial block	Setup values	Only for simulation
Forever loop	Simulation clocks	Must be bounded for hardware

➔ Real-World Examples: Good vs Bad

Non-synthesizable:	Synthesizable:
<pre>initial begin a = 0; #10 a = 1; end</pre>	<pre>always @(posedge clk) a <= a + 1;</pre>

How to Check **Synthesizability?**



Use tools like Yosys, Verilator, or synthesis reports



Avoid constructs flagged as simulation-only



Focus on RTL intent and hardware-mapped logic

Exercise: Fix This Design

Original Code (Non-Synthesizable):

```
initial begin
    x = 0;
    #10 x = 1;
end
```

Fix It:

```
always @(posedge clk)
    x <= 1;
```

Thank You

echiphu

